Solveur de cinématique inverse redondante — Rapport de présentation

Le solveur de cinématique inverse, transforme chaque point d'un centre d'un bushing en une configuration complète et exécutable : six angles du robot ABB, une position de rail et un angle de mandrin sans collision.



Figure 1 – Poste d'enroulement : pièce montée sur mandrin rotatif, LIST.

Ce rapport présente le problème résolu, les notions de robotique nécessaires, les étapes de calcul du solveur et son intégration dans la chaîne de production jusqu'au programme RAPID.

Outils : RobotComponents et Grasshopper (Rhino), composant C# *IK SOLVER*, contrôleur ABB et RobotStudio.

1. Position du problème

Le système dispose de trois réglages de plus que nécessaire pour atteindre une cible : il existe une infinité de solutions, et le solveur doit en trouver une qui soit réellement exécutable.

1.1 Contexte

L'objectif du poste est de déposer un fil de carbone autour d'une série de douilles (*bushings*) fixées sur une pièce, qui peut être montée sur un mandrin rotatif. Le dépôt est réalisé par un robot ABB I monté sur un rail linéaire.

La géométrie du parcours est fournie sous forme d'une liste de plans cibles. Chaque plan définit le point de passage de l'outil et l'orientation qu'il doit avoir. Le solveur convertit chacun de ces plans en une configuration complète pour le contrôleur ABB : six angles articulaires A1 à A6, une position de rail et un angle de mandrin.

1.2 Paramètres disponibles

Paramètre	Symbole	Unité	Rôle
Six axes du bras	q = (q_1 … q_6), notés A1 à A6	degrés	positionnent et orientent l'outil
Rail (axe externe EJa)	E_1	mm	déplace l'ensemble du robot le long de la cellule
Mandrin (axe externe EJb)	θ_b	degrés	fait tourner la pièce
Rotation de l'outil autour de son axe (*yaw*)	ψ	degrés	paramètre libre du procédé

Le *yaw* est libre car le fil est déposé le long de l'axe de l'outil : faire tourner l'outil sur lui-même ne modifie pas le dépôt. Le solveur peut donc le choisir.

1.3 Redondance du système

Placer un outil dans l'espace impose 6 contraintes : 3 en position et 3 en orientation. Le système dispose de 9 paramètres, et l'une des 6 contraintes (le *yaw*) est relâchée. Le nombre de degrés de liberté excédentaires est donc :

$$\dim(\mathcal{S}) = 9 - 6 + 1 = 3$$

Pour une même cible, il n'existe donc pas une solution mais une famille continue de solutions $\mathscr{S}$, dont seule une partie respecte les butées et évite les collisions. C'est ce qui impose une recherche plutôt qu'un calcul direct.

1.4 Formulation

Pour chaque plan cible P_i, le solveur cherche les quatre inconnues (E_1, θ_b, ψ, q) telles que l'outil atteigne la cible, une fois celle-ci entraînée par le mandrin et mise en forme pour le robot :

$$\mathrm{FK}(q, E_1, \theta_b) = \Phi\big(\mathcal{T}(P_i, \theta_b),\ \psi\big)$$

sous les contraintes suivantes :

- chaque axe q_k reste entre ses butées q_k^min et q_k^max ;
- le rail reste dans sa course, $E_1 \in [0 ; 9\,000]$ mm ;
- aucune collision n'est détectée.

Dans cette équation, FK est le modèle géométrique direct du robot (chapitre 2), $\mathscr{T}$ la rotation de la pièce par le mandrin (chapitre 4) et Φ la mise en forme de l'orientation (chapitre 5).

Le solveur ne recherche pas la meilleure solution mais la première solution admissible, en parcourant les candidats dans un ordre de préférence soigneusement construit (chapitre 8). Ce choix est dicté par le coût de calcul : chaque candidat demande un calcul de cinématique inverse et, potentiellement, un test de collision entre maillages, pour plusieurs centaines de cibles par cycle. Grâce à l'ordre de parcours, le premier candidat valide est proche du candidat idéal.

2. Notions de base : cinématique directe et inverse

Calculer la position de l'outil à partir des angles du robot est simple et donne un résultat unique ; l'opération inverse, dont le solveur a besoin, peut donner zéro, plusieurs ou une infinité de réponses.

2.1 Espace articulaire et espace opérationnel

L'état du robot peut se décrire de deux manières :

- l'espace articulaire : les angles des moteurs A1 à A6, complétés ici par le rail E_1 et le mandrin θ_b. C'est ce que le contrôleur commande réellement ;
- l'espace opérationnel : la *pose* (position et orientation) du point de travail de l'outil, le TCP (*Tool Center Point*). C'est dans cet espace que la géométrie d'enroulement est définie.

Le rôle du solveur est de passer du second au premier.

2.2 Cinématique directe (FK)

La cinématique directe répond à la question : « où se trouve l'outil si les moteurs sont à tels angles ? » On l'obtient en enchaînant, de la base vers l'outil, la transformation apportée par chaque segment du bras :

$$X = \mathrm{FK}(q) = T_{base} \cdot A_1(q_1) \cdot A_2(q_2) \cdots A_6(q_6) \cdot T_{outil}$$

Chaque terme $A_k(q_k)$ décrit le passage d'un segment au suivant : il dépend de l'angle du moteur k et de dimensions fixes du robot (longueurs, angles entre axes), déjà intégrées dans le modèle RobotComponents. Le rail ajoute simplement une translation de toute la base le long de son axe :

$$T_{base}(E_1) = \mathrm{Trans}(E_1 \cdot u_{rail}) \cdot T_{base}(0)$$

La cinématique directe est explicite (un produit de matrices), toujours définie et unique. Dans le solveur, elle sert à calculer l'angle de présentation du mandrin (chapitre 7), à positionner les maillages du robot pour le test de collision (chapitre 9) et à l'affichage dans Grasshopper.

2.3 Cinématique inverse (IK)

La cinématique inverse répond à la question opposée : « quels angles donner aux moteurs pour que l'outil atteigne telle pose ? » Le résultat n'est plus une valeur unique mais un ensemble :

$$\mathrm{IK}(X) = \{\, q \;:\; \mathrm{FK}(q) = X \,\}$$

Cet ensemble peut être :

- vide : la pose est hors de portée (trop loin, trop près ou orientation impossible), diagnostic *out of reach* ;
- fini : jusqu'à 8 solutions pour un robot six axes à poignet sphérique ;
- infini : au voisinage d'une singularité, ou lorsque des axes supplémentaires sont ajoutés (rail, mandrin).

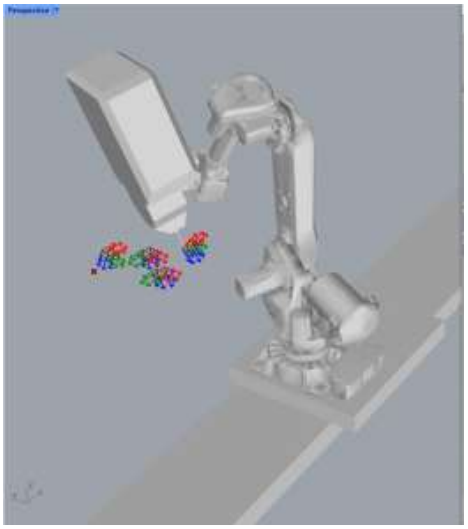
Les 8 solutions correspondent à trois choix binaires de posture :

Choix	Options	Code ABB
Épaule	cible atteinte par la gauche ou par la droite de l'axe 1	cf1
Coude	coude vers le haut ou vers le bas	cf4
Poignet	orientation directe ou poignet retourné (A4 et A6 à +180°, A5 de signe opposé)	cf6

Le contrôleur ABB désigne la posture à utiliser par le confdata [cf1, cf4, cf6, cfx]. Le solveur l'impose à `ConfigurationData(1, 0, 0, 0)` : la solution est ainsi unique et le résultat reproductible d'une exécution à l'autre. Ce choix évite surtout un changement de posture entre deux points voisins, qui provoquerait un mouvement ample et imprévisible du bras.

Pour ce type de robot, la cinématique inverse se calcule par formules analytiques, sans itération. RobotComponents renvoie les angles ainsi que deux verdicts exploités par le solveur : `ErrorText` (hors d'atteinte, singularité, configuration impossible) et `IsInLimits` (respect des butées).

Configuration valide



Configuration en erreur

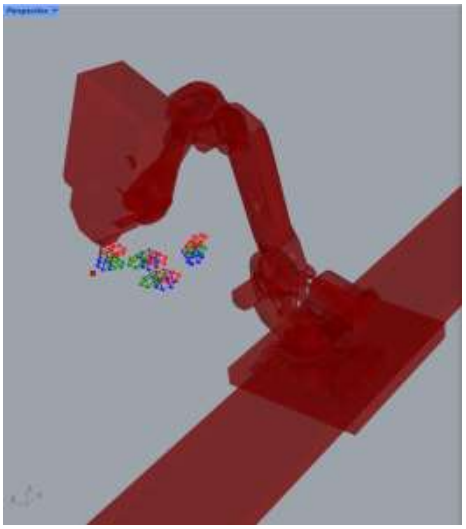


Figure 2 – Affichage du robot dans Grasshopper : à gauche, une configuration cohérente ; à droite (en rouge), une configuration en erreur (singularité, limite d'axe…).

2.4 Butées et enveloppe de travail

Chaque axe a une plage de mouvement limitée. L'ensemble des poses atteignables en respectant ces butées forme l'enveloppe de travail du robot. Elle n'est pas sphérique : elle comporte notamment une zone centrale inaccessible autour de la colonne du robot.

Le solveur applique deux niveaux de butées : celles du constructeur (`IsInLimits`) et celles, plus serrées, définies par l'utilisateur dans `joint_limits`, qui interdisent des postures autorisées mécaniquement mais indésirables.

2.5 Singularités

Une singularité est une posture dans laquelle le robot perd la capacité de se déplacer dans certaines directions. Elle se lit sur la relation entre les vitesses des moteurs et la vitesse de l'outil, donnée par la matrice jacobienne J :

$$\dot{X} = J(q)\,\dot{q} \quad \Longrightarrow \quad \dot{q} = J(q)^{-1}\,\dot{X}$$

Lorsque J n'est plus inversible, les vitesses moteurs nécessaires deviennent très grandes au voisinage de cette posture : le robot s'emballe ou déclenche un arrêt d'urgence, même pour une trajectoire lente. Les singularités doivent donc être évitées avec une marge. On distingue :

- singularité de poignet : A5 = 0, les axes 4 et 6 s'alignent. C'est la plus fréquente, d'où une bande autour de A5 = 0 souvent exclue des butées projet ;
- singularité d'épaule : le centre du poignet passe à la verticale de l'axe 1 ;
- singularité de coude : le bras est entièrement tendu.

> Terminologie ABB. Le message « elbow singularity » renvoyé par ABB désigne le cas où le centre du poignet approche la verticale de l'axe 1, et non un bras trop tendu. Le remède consiste à décaler latéralement le poignet, en jouant sur le rail ou sur le mandrin.

Le solveur ne calcule pas la jacobienne : les singularités lui parviennent sous forme de messages d'erreur de l'IK et sont traitées comme des cas « hors d'atteinte » (chapitre 8).

2.6 Redondance

Un système est redondant lorsqu'il possède plus de degrés de liberté que la tâche n'en impose. À posture fixée, le bras seul admet une solution unique ou aucune. L'ajout du rail, du mandrin et du *yaw* libre donne une famille continue de solutions à trois dimensions, qu'il faut explorer. Toute l'architecture du solveur découle de ce constat.

2.7 Glossaire

Terme	Définition
FK — cinématique directe	angles des moteurs → pose de l'outil ; explicite et unique
IK — cinématique inverse	pose de l'outil → angles des moteurs ; 0, 8 ou une infinité de solutions
TCP	point de l'outil dont on commande la pose
tool0	repère de la bride du robot, avant montage de l'outil
Pose	position et orientation, représentées par un repère ou une matrice 4 × 4
robtarget	instruction ABB décrivant une cible : position, quaternion, confdata, axes externes
Workobject (wobj)	repère de référence dans lequel une cible est exprimée (uframe porteur, oframe pièce)
Axe externe	axe motorisé hors du bras : EJa pour le rail, EJb pour le mandrin
confdata	code ABB désignant l'une des huit postures
Butée articulaire	limite de mouvement d'un axe
Enveloppe de travail	ensemble des poses atteignables par l'outil
Singularité	posture où le robot perd une direction de mouvement
Redondance	excédent de degrés de liberté ; famille continue de solutions
Yaw (ψ)	rotation de l'outil autour de son propre axe
MoveL / MoveJ	mouvement rectiligne de l'outil / mouvement interpolé sur les axes
RAPID	langage de programmation des contrôleurs ABB

3. Repères, transformations et quaternions

Toute pose est représentée par une matrice 4 × 4 qui réunit orientation et position ; les changements de repère s'enchaînent alors par simples multiplications.

3.1 Repères et matrices homogènes

Un repère (objet *Plane* de Rhino) est défini par une origine O et trois axes unitaires et perpendiculaires x, y, z, avec z = x × y. On lui associe la matrice homogène :

$$T = \begin{bmatrix} x_1 & y_1 & z_1 & O_1 \\ x_2 & y_2 & z_2 & O_2 \\ x_3 & y_3 & z_3 & O_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} R & O \\ 0 & 1 \end{bmatrix}$$

- le bloc R (3 × 3) contient les trois axes en colonnes : c'est l'orientation ;
- la colonne O contient l'origine : c'est la position ;
- la dernière ligne n'a pas de contenu géométrique : elle permet de combiner rotation et translation en une seule multiplication.

La composition de deux changements de repère se réduit ainsi à un produit de matrices, et le changement inverse à l'inverse de la matrice.

3.2 Point et direction

Un point subit la rotation et la translation ; une direction (un axe, une normale) ne subit que la rotation :

$$p' = R\,p + O \quad \text{(point)} \qquad\qquad v' = R\,v \quad \text{(direction)}$$

Cette distinction est appliquée dans la fonction `TransformPlaneWobj` : l'origine du plan subit la transformation complète, ses axes seulement la rotation. Sans elle, les axes « dérivent » avec la translation et les orientations deviennent fausses dès que le workobject n'est pas à l'origine du monde.

3.3 Quaternions

Le contrôleur ABB décrit une orientation par un quaternion, quatre nombres qui codent une rotation de manière compacte. Les poses échangées avec le contrôleur ont le format :

```
"px, py, pz, qw, qx, qy, qz"
```

La composante scalaire qw est placée en premier (convention ABB). Le quaternion est renormalisé à la lecture pour corriger les erreurs d'arrondi :

$$q \leftarrow \frac{q}{\|q\|}, \qquad \|q\| = \sqrt{w^2 + x^2 + y^2 + z^2}$$

La conversion inverse, de la matrice vers le quaternion, est assurée par `RotmatToQuat` selon la méthode de Shepperd, qui choisit la formule numériquement la plus stable parmi quatre variantes.

3.4 Principaux symboles

Symbole	Signification	Nom dans le code
$q = (q_1 \dots q_6)$	angles articulaires A1 à A6 (degrés)	RobotJointPosition
E_1	position du chariot sur le rail (mm)	EJa / ext[0]
θ_b	angle du mandrin (degrés)	EJb / ext[1]
θ_teach	angle du mandrin lors de l'apprentissage	iTeachingAngle
ψ	rotation de l'outil autour de la normale (yaw)	yaw
a_0, a	point et direction de l'axe du mandrin	axisOrigin, axisDir
T_a, T_o	poses de l'axe et de la pièce (workobject ABB)	iAxisStr, iOframeStr
T_u(θ_b)	pose de la pièce à l'angle θ_b	ComputeUframePose
n_0	normale de la bushing	nRaw
z_t	axe Z de l'outil à la pose de repos	tcpHome.ZAxis

4. Cinématique du mandrin

Le solveur reproduit à l'identique le calcul du contrôleur ABB pour une pièce portée par un axe externe rotatif ; la concordance a été vérifiée à 0,000 mm.

4.1 Axe de rotation

L'axe du mandrin est la droite passant par le point a_0 et dirigée par le vecteur unitaire a. Tous deux sont lus dans la chaîne `iAxisStr` fournie par le contrôleur ; la direction est l'un des axes de ce repère, choisi par `iMandrelAxis` (l'axe z en pratique).

4.2 Rotation autour d'un axe quelconque

La rotation d'angle δ autour de la direction unitaire a est donnée par la formule de Rodrigues (fonction `AxisAngleRot`) :

$$R(a,\delta) = I\cos\delta + (1-\cos\delta)\,a\,a^{T} + \sin\delta\,[a]_{\times}$$

L'axe ne passant pas par l'origine du monde, on ramène d'abord a_0 à l'origine, on applique la rotation, puis on replace a_0 (fonction `RotationAboutAxis`) :

$$T_{rot}(a_0,a,\delta) = \mathrm{Trans}(a_0)\cdot\mathrm{Rot}(a,\delta)\cdot\mathrm{Trans}(-a_0)$$

Les points situés sur l'axe restent bien immobiles.

4.3 Angle relatif à l'apprentissage

La géométrie de la pièce a été numérisée alors que le mandrin se trouvait à l'angle θ_teach. L'angle effectivement appliqué est donc l'écart à cette référence :

$$\delta(\theta_b) = \theta_b - \theta_{teach}$$

4.4 Pose de la pièce dans la cellule

La pose de la pièce s'obtient en composant trois transformations :

$$T_u(\theta_b) = T_{rot}\big(a_0,a,\delta(\theta_b)\big)\cdot T_a\cdot T_o$$

La lecture se fait de droite à gauche : on part du repère de la pièce T_o (*oframe*) exprimé par rapport à l'axe, on le place dans la cellule par T_a, puis on applique la rotation du mandrin. L'ordre des facteurs est essentiel : une permutation produit un décalage qui n'apparaît que pour certaines positions du workobject.

Pour θ_b = θ_teach, la rotation est nulle et l'on retrouve la pose de référence, utilisée pour ramener les plans de la pièce dans la cellule :

$$T_{ref} = T_a\cdot T_o$$

5. Mise en forme de l'orientation

Chaque plan issu de la modélisation subit trois opérations successives, appliquées une seule fois et toujours dans le même ordre, avant d'être envoyé au robot.

Le plan d'origine n'est pas directement exploitable : le sens de sa normale n'est pas garanti, son axe x n'a pas de signification pour le procédé, et la convention robot impose que l'axe z de l'outil pointe vers la matière. La chaîne appliquée est :

$$\Phi = \mathrm{FixPlaneForRobot} \circ \mathrm{RotatePlane}(\psi) \circ \mathrm{StabilizePlane}$$

5.1 Stabilisation de la normale

La normale z est forcée vers le haut du monde : si elle pointe vers le bas, elle est inversée. L'axe x est ensuite corrigé pour être exactement perpendiculaire à la normale, en lui retirant sa composante le long de z (procédé de Gram-Schmidt) :

$$\tilde{x} = \frac{x - (x \cdot \tilde{z})\,\tilde{z}}{\lVert x - (x \cdot \tilde{z})\,\tilde{z} \rVert}, \qquad \tilde{y} = \tilde{z} \times \tilde{x}$$

Si x est parallèle à la normale, cette correction donne un vecteur nul ; le code utilise alors l'axe X du monde comme axe de secours.

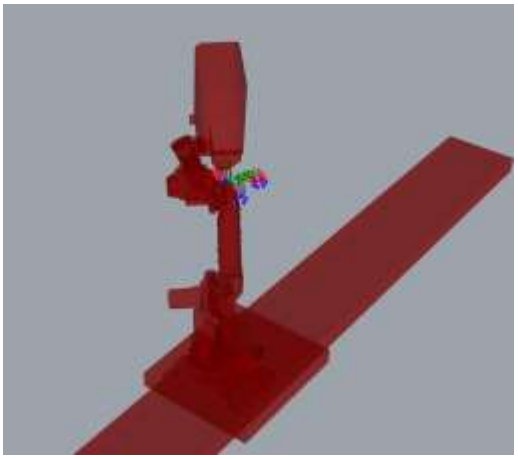
5.2 Rotation de yaw

Les axes x et y tournent d'un angle ψ autour de la normale, qui reste inchangée, tout comme la position :

$$x_\psi = \cos\psi\,\tilde{x} + \sin\psi\,\tilde{y}, \qquad y_\psi = -\sin\psi\,\tilde{x} + \cos\psi\,\tilde{y}, \qquad z_\psi = \tilde{z}$$

Cette rotation ne modifie pas le dépôt du fil, qui se fait le long de la normale, mais elle change fortement la posture du poignet. C'est le levier principal du solveur pour ramener les axes A4, A5 et A6 dans leurs butées.

Avant rotation de yaw



Après rotation de yaw

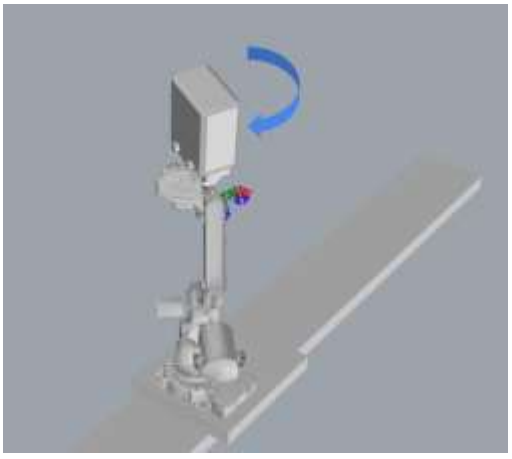


Figure 3 – Effet de la rotation de yaw : la cible est la même, mais la rotation de l'outil autour de son axe (flèche bleue) ramène le poignet dans ses butées.

5.3 Retournement pour le robot

La normale est enfin inversée pour pointer vers la matière, conformément à la convention outil. L'opération revient à conserver x et à inverser y et z :

$$\Phi_{fix}(O, x, y, z) = (O,\ x,\ -y,\ -z)$$

Il s'agit exactement d'une rotation de 180° autour de l'axe x du plan.

6. Placement initial du rail et du yaw

Avant toute recherche, le solveur calcule pour chaque cible une position de rail et un yaw « idéaux » ; la recherche part de ce point et s'en éloigne progressivement, si bien que la qualité de cette estimation conditionne celle du résultat.

6.1 Position du rail

Le robot se place à une distance latérale de la cible, le déport, qui varie linéairement le long de l'axe Y de la pièce : d_min à l'extrémité Y_min, d_max à l'extrémité Y_max. Pour une cible d'origine O = (X_O, Y_O, Z_O) :

1. la coordonnée Y_O est ramenée dans l'intervalle [Y_min ; Y_max], puis convertie en une fraction α comprise entre 0 et 1 :

$$\alpha = \frac{Y_c - Y_{min}}{Y_{max} - Y_{min}} \in [0, 1]$$

2. le déport est interpolé entre ses deux valeurs extrêmes :

$$d(\alpha) = (1 - \alpha)\, d_{min} + \alpha\, d_{max}$$

3. le chariot est placé à cette distance de la cible, du côté fixé par `flipOffsetDirection` (ε = +1 ou −1) :

$$E_1 = X_O + \varepsilon \cdot d(\alpha), \qquad E_1 \in [0; 9000]\ \text{mm}$$

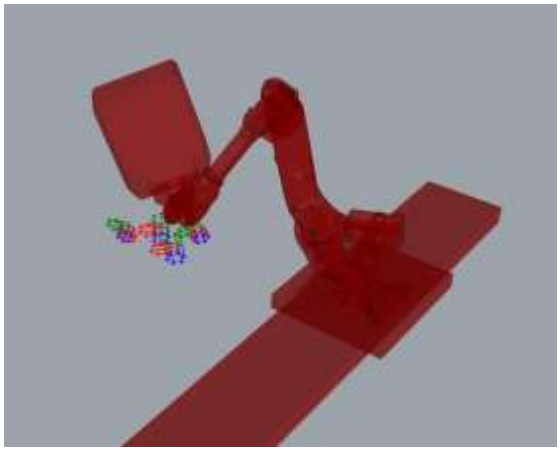
La saturation de l'étape 1 évite des positions de rail aberrantes pour les cibles situées aux extrémités de la pièce.

> Paramètre déterminant. Le signe ε, c'est-à-dire le côté de la pièce où se place le robot, est le paramètre le plus influent du solveur. Du mauvais côté, le robot doit contourner la pièce et la quasi-totalité des cibles deviennent inatteignables.

Les positions de rail sont essayées dans l'ordre suivant :

1. continuité : la position retenue pour la cible précédente, afin de limiter les déplacements du chariot, le temps de cycle et les à-coups ;
2. valeur calculée : celle obtenue ci-dessus ;
3. secours : un balayage autour de la valeur calculée, par pas de 200 mm jusqu'à ±1 400 mm.

Position de rail inadaptée



Rail repositionné

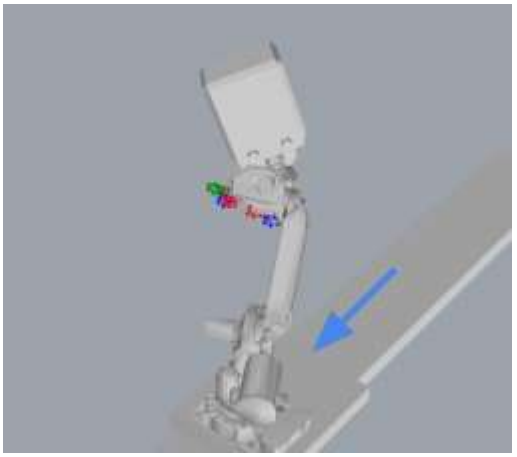


Figure 4 – Effet du déplacement du rail : en déplaçant le chariot (flèche bleue), le solveur ramène la cible dans l'enveloppe de travail du robot.

6.2 Balayage en spirale

Pour tester des valeurs autour d'un point de départ, le solveur alterne de part et d'autre en s'éloignant progressivement, avec un pas s et une amplitude maximale R_max :

$$\mathcal{S}(s, R_{max}) = (0,\ +s,\ -s,\ +2s,\ -2s,\ \dots)$$

Pour le rail : 0, +200, −200, +400, −400 … jusqu'à ±1 400 mm. Les candidats sont ainsi énumérés par distance croissante au point de départ : le premier candidat valide est donc aussi l'un des plus proches de l'idéal. Ce principe est utilisé pour le rail, le yaw et le mandrin.

6.3 Yaw idéal

Le yaw de départ est celui qui oriente l'axe x de la cible vers la base du robot, posture généralement la plus ouverte. On projette la direction cible → base B sur le plan de la cible, puis on mesure l'angle signé entre l'axe x et cette direction :

$$D = B - O, \qquad u = \frac{D - (D \cdot z)\, z}{\lVert D - (D \cdot z)\, z \rVert}$$

$$\psi^* = \operatorname{sgn}\big(z \cdot (x \times u)\big) \cdot \arccos\big(\operatorname{clamp}(x \cdot u,\ -1,\ 1)\big)$$

Le produit scalaire est borné entre −1 et 1 (*clamp*), car une erreur d'arrondi peut donner une valeur légèrement supérieure à 1, pour laquelle arccos n'est pas défini. Si la normale pointe exactement vers la base, la direction projetée est nulle et le solveur retient ψ* = 0.

Le premier yaw essayé n'est pas ψ* mais le plan d'origine, sans rotation. Le solveur ne réoriente l'outil que si cette pose est inaccessible, en balayant ensuite en spirale autour de ψ*. Ce choix respecte au mieux l'orientation définie par la géométrie et réduit fortement le temps de calcul sur les pièces simples, dont les points sont coplanaires.

7. Angle de présentation du mandrin

Lorsque l'option `iUseTheta` est activée, le solveur calcule l'angle de mandrin qui présente chaque bushing face à l'outil, puis teste 13 angles autour de cette valeur.

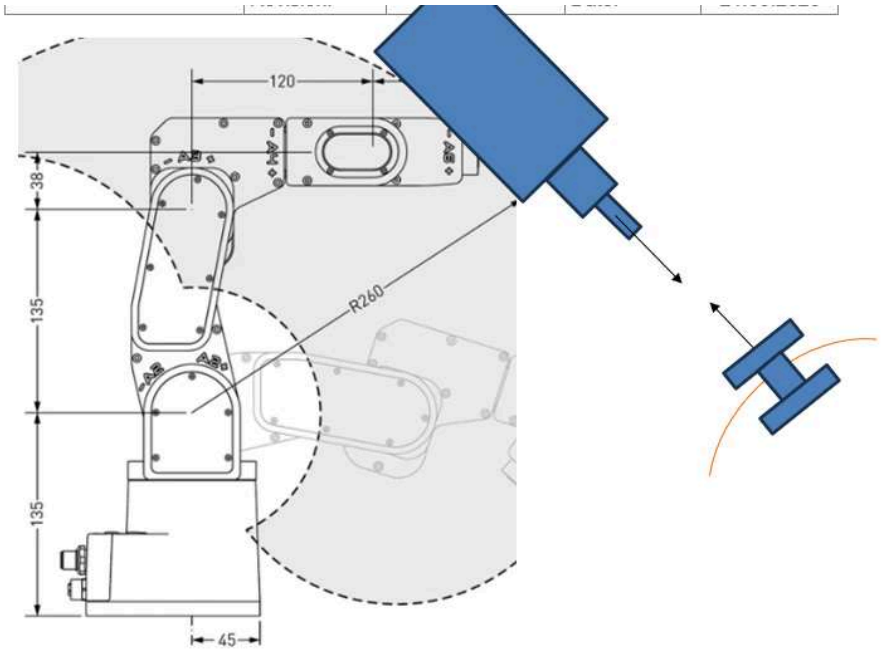


Figure 5 – Principe : le mandrin tourne pour que la normale de la bushing soit alignée avec l'axe de l'outil.

7.1 Critère d'alignement

Deux directions sont comparées :

- n_0 : la normale de la bushing, ramenée dans la cellule par la transformation de référence T_ref ;
- z_t : l'axe z de l'outil lorsque le robot est à sa pose de repos (`iHomeJoints`), obtenu par cinématique directe.

Lorsque le mandrin tourne d'un angle τ, la normale devient $R(a, \tau)\cdot n_0$. Elle garde un angle constant avec l'axe du mandrin et décrit donc un cône autour de celui-ci (figure 6).

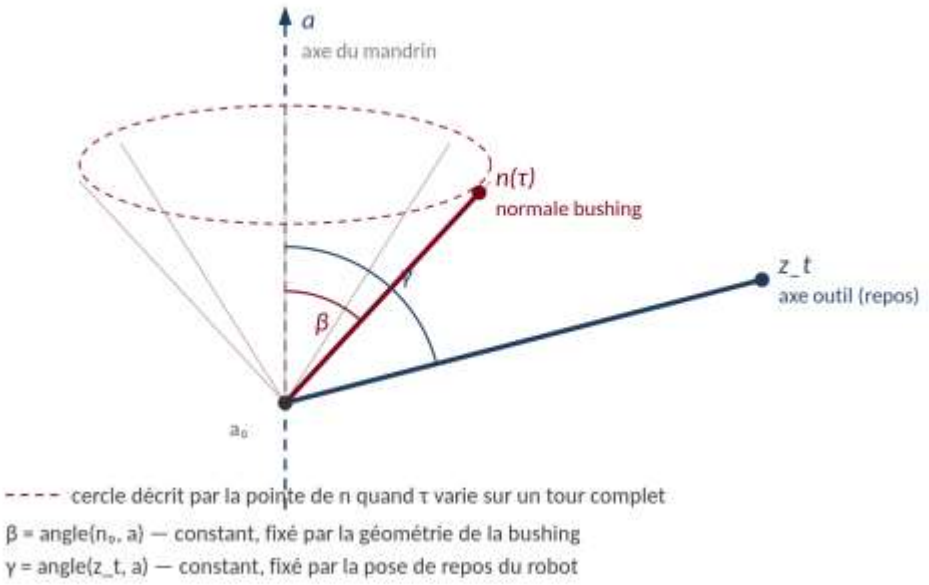


Figure 6 – Lors de la rotation du mandrin, la normale n(τ) de la bushing décrit un cône autour de l'axe a ; β est fixé par la géométrie de la bushing, γ par la pose de repos du robot.

L'angle recherché est celui qui rend la normale et l'axe de l'outil opposés, de sorte que la bushing soit orientée face à l'outil. Le produit scalaire de deux vecteurs unitaires valant −1 lorsqu'ils sont opposés, on maximise :

$$\tau^* = \arg\max_{\tau} f(\tau), \qquad f(\tau) = -\big(R(a,\tau)\, n_0\big) \cdot z_t$$

La valeur f = +1 correspond à un alignement parfait. L'angle absolu du mandrin, qui sert de point de départ (« graine ») à la recherche, s'obtient en ajoutant l'angle d'apprentissage :

$$\theta_s = \tau^* + \theta_{teach}$$

7.2 Résolution par balayage

La fonction f est évaluée sur 360 angles, de −180° à +179° par pas de 1°, et la meilleure valeur est retenue :

$$\tau^* \approx \arg\max\{\, f(\tau) : \tau = -180^\circ + k \cdot 1^\circ,\ k = 0 \ldots 359 \,\}$$

Le coût est négligeable devant un seul calcul de cinématique inverse, et la précision de ±0,5° est largement suffisante, puisque cette valeur sert uniquement de point de départ à un balayage par pas de 5°.

7.3 Angles de mandrin testés

Autour de la graine θ_s, le solveur teste des angles en spirale, dans une fenêtre W et avec un pas h :

$$\mathcal{C} = (\theta_s,\ \theta_s + h,\ \theta_s - h,\ \theta_s + 2h,\ \ldots) \cap [\theta_s - W,\ \theta_s + W], \qquad |\mathcal{C}| = 2\,\lfloor W/h \rfloor + 1$$

Avec les valeurs par défaut W = 30° et h = 5°, cela représente 13 angles.

> Stratégie centrée. Chaque bushing repart de son propre angle idéal θ_s, plutôt que de l'angle retenu pour la bushing précédente. Le mandrin peut tourner davantage entre deux bushings, mais chaque

> normale est correctement réalignée sur l'outil : les postures du robot sont plus confortables et le taux de résolution est supérieur. Une approche par enchaînement propagerait au contraire les compromis d'une bushing à toutes les suivantes.

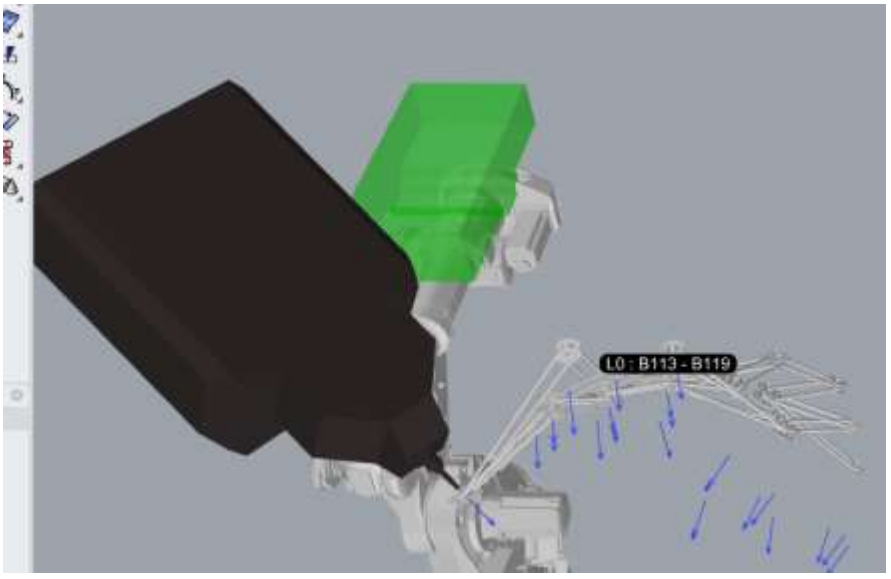


Figure 7 – Simulation de la pièce Novacom2 avec le mandrin. En bleu, les normales des bushings après rotation du mandrin : le solveur aligne l'axe Z de l'outil sur ces normales.

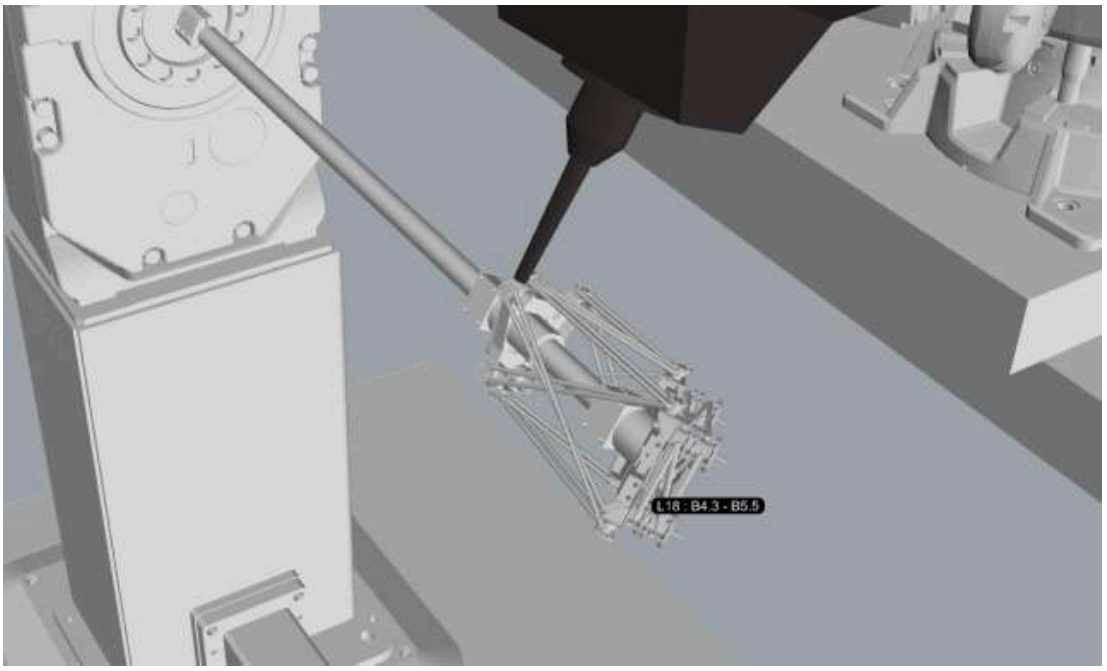


Figure 8 – Simulation des positions du robot et de la rotation du mandrin pour une pièce du satellite Goffset.

8. Algorithme de recherche

Le solveur parcourt les combinaisons (mandrin, rail, yaw) par ordre de préférence décroissante et retient la première qui satisfait les quatre conditions d'acceptation, dans la limite de 400 essais par cible.

8.1 Conditions d'acceptation

Un candidat (θ_b, E_1, ψ) est accepté si et seulement si les quatre conditions sont réunies :

$$\mathrm{Accept}(\theta_b, E_1, \psi) = [\,\mathscr{E} = \varnothing\,] \wedge [\,\mathrm{IsInLimits}\,] \wedge [\,q \in \mathscr{Q}\,] \wedge [\,\neg\,\mathrm{Collision}\,]$$

1. $\mathscr{E} = \varnothing$: la cinématique inverse ne renvoie aucune erreur (hors d'atteinte, singularité, configuration impossible) ;
2. IsInLimits : les butées du modèle constructeur sont respectées ;
3. $q \in \mathscr{Q}$: chaque axe respecte les butées projet, $q_k^{min} \le q_k \le q_k^{max}$ pour k = 1 à 6 ;
4. pas de collision entre l'outil et le boîtier du robot (chapitre 9).

Les conditions 2 et 3 ne sont pas redondantes : la première applique les limites du modèle de robot, la seconde des limites plus serrées choisies par l'utilisateur (éloignement de la singularité de poignet, contraintes de câblage, etc.).

8.2 Structure des boucles

Pour chaque cible, le solveur calcule d'abord les valeurs initiales de rail, de yaw et de mandrin (chapitres 6 et 7), puis parcourt trois boucles imbriquées, du paramètre le plus contraint au plus libre :

1. angle de mandrin (13 valeurs) : la pièce est tournée, la cible recalculée dans la cellule, ainsi que le rail et le yaw initiaux ;
    1. position de rail (16 valeurs : continuité, valeur calculée, 14 valeurs en spirale) ; les positions hors course sont ignorées ;
        1. yaw (12 essais au plus par angle de mandrin) : mise en forme de l'orientation (chapitre 5), calcul de cinématique inverse, puis vérification des conditions d'acceptation.

Dès qu'un candidat est accepté, le solveur sort des trois boucles et passe à la cible suivante.

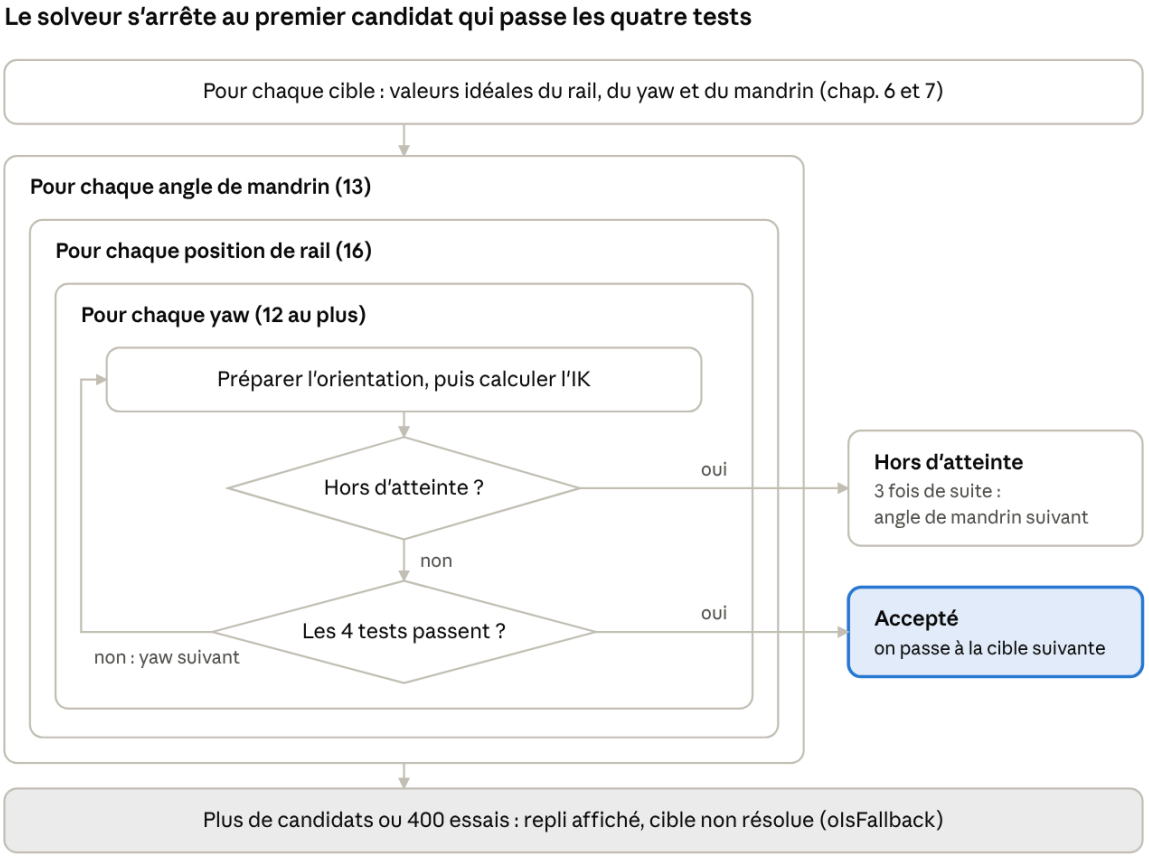


Figure – Structure de la recherche pour une cible : trois boucles imbriquées et deux tests.

Un échec aux tests fait passer au yaw suivant ; trois « hors d'atteinte » de suite font passer directement à l'angle de mandrin suivant.

8.3 Interruption sur « hors d'atteinte »

La rotation de yaw ne modifie pas la position du TCP, seulement son orientation. La distance entre la cible et la base du robot est donc indépendante de ψ :

$$\|\Phi(P,\psi).O - B\| \text{ indépendant de } \psi$$

Or un diagnostic « hors d'atteinte » traduit essentiellement un problème de distance. Si trois yaws consécutifs renvoient ce diagnostic, les suivants le renverront très probablement aussi : le solveur interrompt la boucle de yaw et passe à l'angle de mandrin suivant, qui déplace réellement la cible. Ce seuil est réglé par `iOutOfReachTol`.

Un seuil de trois plutôt que d'un laisse une marge pour les cas proches de la limite d'enveloppe, où l'orientation du poignet influe légèrement sur l'atteignabilité.

8.4 Nombre d'essais

Le nombre d'appels de cinématique inverse par cible est borné par :

$$N \le \min\left(13 \times 16 \times 12,\ \ \text{iMaxAttempts}\right) = \min(2\,496,\ 400)$$

Le plafond global de 400 essais est donc atteint bien avant la borne combinatoire. La sortie `oAttempts` se lit comme un indicateur de marge : peu d'essais signifient une cible facilement atteignable, beaucoup d'essais une cible en limite.

9. Détection de collision outil / boîtier

Le modèle de robot fourni par RobotComponents ne comporte pas le boîtier électrique monté sur l'avant-bras, principal obstacle pour un outil d'enroulement long ; le solveur intègre donc son maillage et vérifie l'absence d'interférence avant d'accepter une pose.

Boîtier sur le robot réel

Collision détectée en simulation

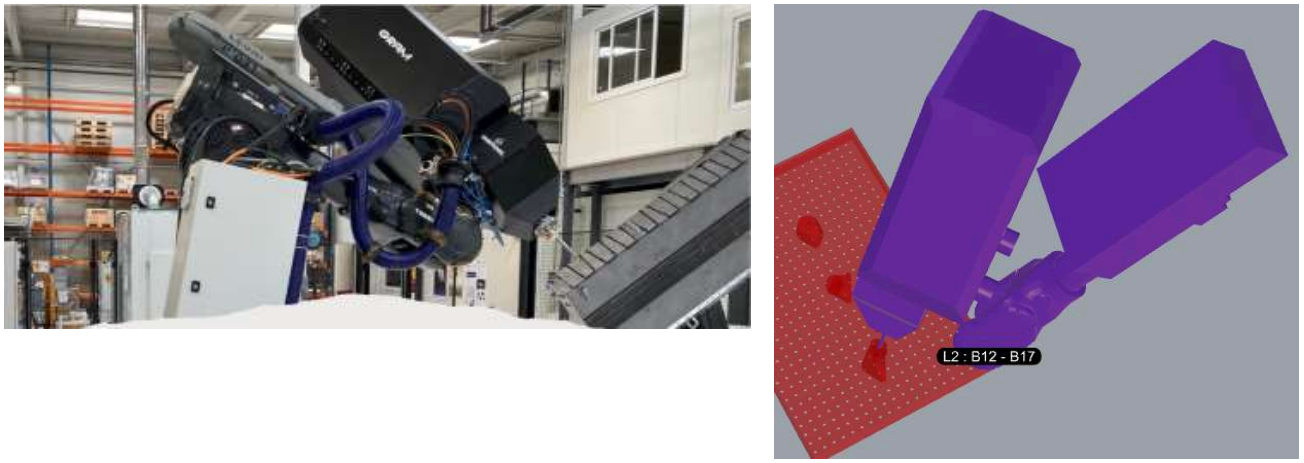


Figure 9 – À gauche, le boîtier électrique monté sur l'avant-bras du robot ; à droite, une configuration où l'outil entre en collision avec ce boîtier.

9.1 Repositionnement entre robots

Le maillage du boîtier a été relevé sur le robot de calibration (TERRA). Les cinq robots de la cellule étant du même modèle, le boîtier occupe la même position par rapport à leur base. Une simple transformation de repère à repère suffit pour le placer sur n'importe quel robot :

$$T_{reloc} = T_{robot} \cdot T_{calib}^{-1}, \qquad \text{boîtier}_{courant} = T_{reloc} \cdot \text{boîtier}_{calib}$$

Sur le robot TERRA, T_reloc est l'identité. Ce mécanisme évite de recaler manuellement le maillage pour chacun des cinq robots.

9.2 Suivi du boîtier par l'avant-bras

Le boîtier doit suivre rigidement l'avant-bras (segment J3). Le maillage de J3 étant connu à la pose de référence et à la pose testée, avec les mêmes sommets dans le même ordre, trois points non alignés suffisent pour retrouver exactement le déplacement :

1. le centroïde c du maillage ;
2. le sommet le plus éloigné du centroïde, pour une bonne précision angulaire ;
3. le sommet qui forme avec les deux premiers le triangle d'aire maximale, ce qui garantit que les trois points ne sont pas alignés.

Ces trois points définissent un repère F_ref à la pose de référence et un repère F_cur à la pose courante. La transformation appliquée au boîtier est :

$$T = T_{cur} \cdot T_{ref}^{-1}$$

I

9.3 Test d'interférence en deux phases

L'intersection de deux maillages étant coûteuse, elle est précédée d'un rejet rapide par boîtes englobantes alignées sur les axes du monde :

$$\text{Phase 1 : } BB(A) \cap BB(B) = \varnothing \;\Rightarrow\; \text{pas de collision}$$

$$\text{Phase 2 : } \mathcal{L} = \mathrm{MeshMeshFast}(A, B), \qquad \text{Collision} \iff |\mathcal{L}| > 0$$

La phase 1 se réduit à trois comparaisons d'intervalles et écarte la grande majorité des configurations. La phase 2 calcule l'ensemble $\mathscr{L}$ des segments d'intersection entre les deux surfaces : la collision est avérée dès qu'il en existe au moins un.

Une pose en collision n'est pas invalide au sens de la cinématique : elle est simplement rejetée, et le solveur poursuit avec le yaw suivant pour trouver une orientation d'outil qui dégage le boîtier. Le drapeau `oCollision` signale qu'au moins une pose par ailleurs valide a été écartée pour cette raison.

10. Réglages et sorties du composant

Le composant expose ses paramètres de réglage et un ensemble de sorties de diagnostic ; les paramètres qui fixent le point de départ de la recherche (côté d'implantation, pose de repos, déport) ont un effet bien supérieur à ceux qui en fixent l'étendue.

10.1 Principales entrées

Entrée	Défaut	À quoi elle sert
iPlanesWobj	—	les plans cibles, dans le repère de la pièce
iRob	—	le modèle du robot (dimensions, butées, position de la base)
joint_limits	—	les butées du projet pour A1 à A6
flipOffsetDirection	— (oui/non)	de quel côté de la pièce se place le robot
Y_min / Y_max	—	zone de la pièce sur laquelle le déport varie
offset_min / offset_max	—	déport minimal et maximal (d_min, d_max)
iAxisStr / iOframeStr	—	position de l'axe du mandrin et de la pièce (workobject ABB)
iTeachingAngle	—	angle du mandrin lors de la numérisation (θ_teach)
iMandrelAxis	z	axe de rotation du mandrin
iUseTheta	false	active le calcul de l'angle de mandrin face à l'outil
iEjbSeed	−30	angle de mandrin de départ si iUseTheta est désactivé
iEjbWindow / iEjbStep	30 / 5	fenêtre et pas de recherche du mandrin (degrés)
YAW_STEP / YAW_MAX_RANGE	—	pas et amplitude de la recherche du yaw
iHomeJoints	0, 0, 0, 0, 30, 0	pose de repos, qui sert à calculer l'axe de l'outil (z_t)
iJ4Box / iCalibPP	—	maillage du boîtier et repère de calibration
iMaxIkPerEjb	12	nombre maximal d'essais par angle de mandrin
iOutOfReachTol	3	nombre de « hors d'atteinte » de suite avant d'abandonner un angle de mandrin
iMaxAttempts	400	nombre maximal d'essais par cible

10.2 Sorties de diagnostic

Une vérification visuelle ne détecte pas une erreur de 180° qui se compense, ni un décalage dans l'arborescence des données. Les sorties ci-dessous permettent de contrôler le résultat par des valeurs.

Sortie	Comment la lire
oSolved	vrai/faux par cible : le premier indicateur à regarder
oAttempts	nombre d'essais utilisés : indique la marge de la solution
oE1_base / oE1_final	rail idéal et rail retenu : leur écart montre l'effort de recherche
oYaw_base / oYaw_final	yaw idéal et yaw retenu : un grand écart signale un mauvais point de départ

Sortie	Comment la lire
oEJb_final	angle de mandrin retenu
oIsFallback	vrai si aucune solution valable n'a été trouvée
oCollision	vrai si une posture valable a été rejetée à cause du boîtier
oPlaneSrc / oPlaneTarget	plan d'origine et plan après préparation : pour vérifier le retournement
debug	détail complet de la recherche pour chaque cible

11. De la pose au robtarget

La finalité du solveur est la production d'un module RAPID exécutable par le contrôleur ABB : chaque cible y est décrite par un robtarget, dont les quatre blocs correspondent aux quatre décisions prises par le solveur.

11.1 Structure du robtarget

```
VAR robtarget <nom> := [ [ x, y, z ],
                         [ q1, q2, q3, q4 ],
                         [ cf1, cf4, cf6, cfx ],
                         [ eax_a, eax_b, eax_c, eax_d, eax_e, eax_f ] ];
```

Bloc RAPID	Contenu	Origine
trans — [x, y, z]	origine du plan cible (mm)	géométrie d'enroulement, inchangée par le solveur
rot — [q1 … q4]	quaternion de l'orientation, scalaire en tête	mise en forme Φ du chapitre 5, yaw ψ inclus
robconf — [cf1, cf4, cf6, cfx]	posture parmi les huit	imposée : [1, 0, 0, 0]
extax — [eax_a … eax_f]	positions des axes externes	eax_a = E_1 (rail), eax_b = θ_b (mandrin), autres à 9E9

La valeur conventionnelle 9E9 indique au contrôleur qu'un axe externe n'est pas utilisé. Exemple de déclaration produite par la chaîne :

```
VAR robtarget Task_101_1_0 :=
    [[787.38, 514.93, 53.01],
     [0.081038, -0.117485, 0.988146, 0.056556],
     [1, 0, 0, 0],
     [5950, 9E9, 9E9, 9E9, 9E9, 9E9]];
```

Le rail est placé à 5 950 mm, la posture est celle imposée et le quaternion code l'orientation issue de la chaîne Φ. Le mandrin n'est pas utilisé dans cet exemple (eax_b = 9E9) ; dans la configuration avec mandrin, cette case porte l'angle θ_b retenu.

11.2 Extraction des sept nombres

Le passage du plan retenu P_fix aux sept nombres du robtarget est direct :

$$(x, y, z) = O(P_{fix}), \qquad (q_1, q_2, q_3, q_4) = \mathrm{quat}\big([\,\hat{x}\ \hat{y}\ \hat{z}\,]\big)$$

Le quaternion est extrait de la matrice dont les colonnes sont les trois axes du plan, par la méthode de Shepperd (§3.3), en respectant la convention ABB scalaire en tête.

11.3 Rôle du workobject

Une même cible peut s'écrire dans le repère de la cellule ou dans celui de la pièce, les deux écritures étant reliées par la transformation du workobject :

$$X_{monde} = T_{wobj} \cdot X_{local} \quad \Longleftrightarrow \quad X_{local} = T_{wobj}^{-1} \cdot X_{monde}$$

Le solveur travaille en interne avec un workobject identité, la rotation du mandrin étant déjà appliquée géométriquement par T_u(θ_b). Les deux écritures n'ont pas le même comportement à l'exécution :

Écriture	Coordonnées écrites	Comportement du contrôleur
Workobject identité	repère de la cellule, figées à l'angle θ_b du calcul	la cible ne suit pas le mandrin ; écriture utilisée pour la simulation, jamais implantée dans le contrôleur
Workobject sur unité mécanique	repère de la pièce, constantes	le contrôleur recalcule le workobject en continu à partir de la position réelle du mandrin ; la cible suit la pièce

La seconde écriture permet le mouvement coordonné : le contrôleur planifie une trajectoire unique pour le bras et les axes externes, de sorte que le TCP suive un segment rectiligne dans le repère de la pièce pendant sa rotation.

11.4 Condition de cohérence

Le contrôleur ne reçoit pas les angles articulaires : il reçoit une pose et résout lui-même la cinématique inverse. La validité du programme repose donc sur l'égalité :

$$\mathrm{FK}_{contrôleur}\big(\mathrm{IK}_{contrôleur}(X)\big) = X = \mathrm{FK}_{solveur}(q, E_1, \theta_b)$$

Cette égalité n'est garantie que si quatre données sont strictement identiques entre la simulation et la cellule :

1. l'outil : le tooldata du contrôleur doit reproduire exactement le TCP déclaré dans Grasshopper ;
2. le workobject : celui sous lequel les cibles ont été écrites ;
3. la configuration : le confdata sous lequel la solution a été trouvée, faute de quoi le contrôleur choisira une autre des huit solutions ;
4. les axes externes : E_1 et θ_b aux valeurs exactes du calcul.

Une divergence sur l'un de ces points produit un programme syntaxiquement valide et correctement simulé, mais faux à l'exécution.

11.5 Informations extérieures au robtarget

Un robtarget décrit un point, pas un mouvement. Trois informations sont définies séparément :

- le type d'interpolation : MoveL (trajectoire rectiligne, retenue pour la dépose continue) ou MoveJ (interpolation articulaire) ;
- la vitesse (*speeddata*) : vitesse du TCP, de l'orientation, des axes externes linéaires et rotatifs ;
- la précision de passage (*zonedata*) : arrondi autorisé au passage d'une cible.

> Réglage de la vitesse du mandrin. En mouvement coordonné, le bras et les axes externes terminent leur mouvement ensemble. Si le TCP parcourt une faible distance alors que le mandrin tourne de plusieurs dizaines de degrés, l'axe rotatif accélère fortement. La stratégie centrée du §7.3 pouvant produire ces rotations, la vitesse des axes rotatifs est limitée par la composante v_reax du speeddata et, pour un montage à forte inertie, l'accélération par l'instruction AccSet.

11.6 Place dans la chaîne de production

1. CAO et table d'enroulement
2. Plans des centres de bushings (un plan par bushing)
3. Solveur IK : E_1, θ_b et ψ pour chaque centre
4. Extrapolation circulaire : n points autour de chaque bushing
5. Génération RAPID : déclarations robtarget (données) et séquence d'enroulement (logique)
6. RobotStudio → contrôleur → FlexPendant

Bloc Grasshopper	Rôle	Relation au solveur
Robot position	implantation du robot dans la cellule	amont : fixe le plan de base et le côté d'implantation
Z-plane offset / Strategy	placement vertical des points sur la bushing	amont : fixe la position des cibles
Rotate Planes	orientation des plans cibles	amont immédiat : alimente la chaîne Φ
Solver determination	résolution des poses aux centres de bushings	le solveur présenté ici
Robot Targets	extrapolation autour des centres et écriture des robtargets	aval direct
Winding Syntax	ordre de dépose des fibres	aval
Validation par visualisation	contrôle du mouvement du robot	utilise les sorties d'affichage du solveur

Le solveur est sans mémoire de procédé : il reçoit une liste de plans et rend une liste de configurations, cible par cible, sans connaître ni la hauteur de la cible sur la bushing ni l'ordre de passage.

En amont, seul le plan central de chaque bushing, situé à mi-hauteur, est transmis au solveur. Les points répartis autour de la douille en sont déduits ensuite par extrapolation circulaire. L'outil restant aligné sur la normale du plan cible tout au long du parcours, le yaw déterminé au centre reste valable pour ces points. Le nombre de points par bushing, qui dépend de sa forme, n'influe ni sur l'entrée ni sur le coût du solveur.

En aval, la *winding syntax* fixe l'ordre de dépose des fibres selon une hiérarchie stricte :

- les liens inférieurs sont enroulés avant les liens supérieurs ;
- les bushings empilées sont traitées de bas en haut ;
- sur une bushing, les liens internes sont achevés en premier ;
- les demi-cycles sont évités.

> Séparation des responsabilités. La syntaxe d'enroulement traite les collisions fibre / pièce ; le solveur traite les collisions outil / robot et l'atteignabilité mécanique. Ces deux problèmes sont disjoints et résolus par des blocs différents.

Le module généré sépare les données (positions cibles) de la logique (procédures d'enroulement). Lorsqu'un paramètre géométrique change (rotation d'un plan, position du rail, angle de mandrin), seules les déclarations de cibles sont régénérées. Le solveur produit exclusivement des données : il garantit que

chaque cible est atteignable, dans les butées et sans collision, avec les valeurs d'axes externes sous lesquelles cette garantie a été établie.

12. Synthèse

Le solveur résout un problème de cinématique inverse comportant trois degrés de liberté excédentaires en le ramenant à une recherche ordonnée sur trois paramètres : l'angle du mandrin, la position du rail et le yaw de l'outil.

Son efficacité repose sur trois principes :

1. un point de départ calculé géométriquement pour chaque paramètre libre : déport interpolé pour le rail, outil orienté vers la base pour le yaw, bushing présentée face à l'outil pour le mandrin ;
2. un balayage en spirale qui teste les alternatives par distance croissante à ce point de départ, de sorte que le premier candidat valide est aussi un bon candidat ;
3. une chaîne d'orientation stricte, appliquée une seule fois, qui garantit la cohérence entre la géométrie modélisée, la convention outil du robot et le workobject écrit dans le programme RAPID.

Pour chaque cible, le solveur fournit ainsi une configuration atteignable, dans les butées et sans collision, directement exploitable par le contrôleur ABB.